

Aula 13 - Testes Unitários com JUnit 5 e Mockito

Duração: 4 horas (1h teórica + 3h prática)

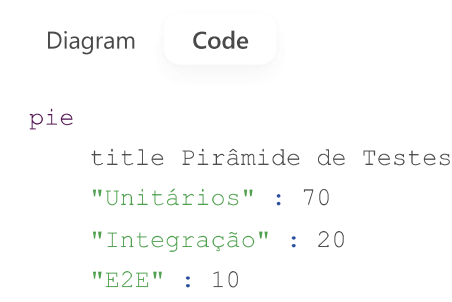
Parte Teórica (1 hora)

1. Revisão da Aula Anterior (15 min)

- Discussão sobre:
 - Upload de arquivos e paginação.
 - Dúvidas sobre `MultipartFile` e `Pageable`.

2. Fundamentos de Testes Unitários (25 min)

Pirâmide de Testes



JUnit 5 vs JUnit 4

Feature	JUnit 5	JUnit 4
Anotações	@Test , @BeforeEach	@Test , @Before
Assertivas	Assertions.assertThrows()	Assert.assertThrows()
Suporte a Lambda	Sim	Não

3. Mockito na Prática (20 min)

Principais Funcionalidades

- `Mockito.mock()` : Cria um mock de uma classe.
- `when().thenReturn()` : Define comportamento do mock.

- `verify()` : Verifica interações com o mock.

Exemplo Básico

java

```
UserRepository mockRepo = Mockito.mock(UserRepository.class);
when(mockRepo.findById(1L)).thenReturn(Optional.of(new User("Admin")));
```

Parte Prática (3 horas)

Atividade: Testando Services e Controllers

Passo 1: Configuração do Ambiente (30 min)

1. Adicionar dependências no `pom.xml` :

xml

```
<dependency>
  <groupId>org.junit.jupiter</groupId>
  <artifactId>junit-jupiter-engine</artifactId>
  <scope>test</scope>
</dependency>
<dependency>
  <groupId>org.mockito</groupId>
  <artifactId>mockito-junit-jupiter</artifactId>
  <scope>test</scope>
</dependency>
```

2. Estrutura de pastas:

plaintext

```
src/test/java/
├── com.example.demo
│   ├── service
│   └── controller
```

Passo 2: Testar Service com Mockito (1h)

1. Classe de teste (`ProductServiceTest.java`):

java

```

@ExtendWith(MockitoExtension.class)
class ProductServiceTest {
    @Mock
    private ProductRepository repository;

    @InjectMocks
    private ProductService service;

    @Test
    void shouldReturnProductWhenValidId() {
        // Arrange
        Product mockProduct = new Product("Notebook", 4500.0);
        when(repository.findById(1L)).thenReturn(Optional.of(mockProduct));

        // Act
        Product result = service.findById(1L);

        // Assert
        assertEquals("Notebook", result.getName());
        verify(repository, times(1)).findById(1L);
    }
}

```

2. Teste para cenário de exceção:

```

java

@Test
void shouldThrowExceptionWhenInvalidId() {
    when(repository.findById(99L)).thenReturn(Optional.empty());
    assertThrows(EntityNotFoundException.class, () -> service.findById(99L));
}

```

Passo 3: Testar Controller com MockMvc (1h)

1. Classe de teste (ProductControllerTest.java):

```

java

@WebMvcTest(ProductController.class)
class ProductControllerTest {
    @Autowired
    private MockMvc mockMvc;

    @MockBean
    private ProductService service;

    @Test
    void shouldReturn200WhenProductExists() throws Exception {
        when(service.findById(1L))
            .thenReturn(new Product("Notebook", 4500.0));
    }
}

```

```

        mockMvc.perform(get("/api/products/1"))
            .andExpect(status().isOk())
            .andExpect(jsonPath("$.name").value("Notebook"));
    }
}

```

2. Teste POST com validação:

java

```

@Test
void shouldReturn400WhenInvalidProduct() throws Exception {
    String invalidProductJson = "{ \"name\": \"\", \"price\": -1 }";

    mockMvc.perform(post("/api/products")
        .contentType(MediaType.APPLICATION_JSON)
        .content(invalidProductJson))
        .andExpect(status().isBadRequest());
}

```

Passo 4: Testes de Integração (30 min)

1. Classe ProductRepositoryTest.java :

java

```

@DataJpaTest
class ProductRepositoryTest {
    @Autowired
    private TestEntityManager entityManager;

    @Autowired
    private ProductRepository repository;

    @Test
    void shouldFindProductsByPriceRange() {
        entityManager.persist(new Product("Mouse", 50.0));
        entityManager.persist(new Product("Teclado", 120.0));

        List<Product> result = repository.findByPriceBetween(100.0, 200.0);
        assertEquals(1, result.size());
    }
}

```

Tarefa para Casa

1. Cobertura de Testes:

- Implementar testes para 100% dos métodos em `ProductService`.

2. Teste de Exceções Customizadas:

- Criar teste para `DataIntegrityViolationException`.

Próxima Aula

- **Tópico:** Testes de Integração com Testcontainers.
- **Atividade prática:** Testar API com banco de dados real em container Docker.

Resultado Esperado

- Cobertura básica de testes para:
 - Services (com Mockito).
 - Controllers (com MockMvc).
 - Repositories (com `@DataJpaTest`).

plaintext

- ✅ 80%+ de cobertura em `ProductService`
- ✅ Testes de controller com validação
- ✅ Pronto para testes de integração avançados

<https://martinfowler.com/articles/practical-test-pyramid/testPyramid.png> (*Pirâmide de Testes - Martin Fowler*) 🚀